



BOLETÍN DE EJERCICIOS UD 7

Utilización avanzada de clases

1. Programar una clase de nombre Hora que nos permita almacenar una hora. Tendrá las siguientes propiedades y métodos:

Propiedades: (protected)

int hora
int minutos
int segundos

Métodos de la clase (públicos):

- Método constructor por defecto, que le de valores aleatorios a las propiedades. La hora será un valor comprendido entre 0 y 23, los minutos y segundos entre 0 y 59.
- Método constructor al que se le pasen como argumentos 3 enteros y se los asigne a las propiedades. Si la hora pasada es incorrecta pondrá la hora a 00:00:00.
- Métodos set y get para todas las propiedades.
- Método de nombre visualizaHora que devuelva un String con la hora en el formato: hh:mm:ss

```
import java.util.Random;

public class Hora {

    protected int hora;
    protected int minutos;
    protected int segundos;

    public Hora() {
        Random r = new Random();
        this.hora = r.nextInt(24);
        this.minutos = r.nextInt(60);
        this.segundos = r.nextInt(60);
    }

    public Hora(int hora, int minutos, int segundos) {
        if (hora < 0 || hora > 23 || minutos < 0 || minutos > 59 || segundos
        < 0 || segundos > 59) {
            this.hora = 0;
            this.minutos = 0;
            this.segundos = 0;
        } else {
            this.hora = hora;
            this.minutos = minutos;
        }
    }
}
```



```
        this.segundos = segundos;
    }

    public int getHora() {
        return hora;
    }

    public void setHora(int hora) {
        this.hora = hora;
    }

    public int getMinutos() {
        return minutos;
    }

    public void setMinutos(int minutos) {
        this.minutos = minutos;
    }

    public int getSegundos() {
        return segundos;
    }

    public void setSegundos(int segundos) {
        this.segundos = segundos;
    }

    public String visualizaHora() {
        return String.format("%02d:%02d:%02d", hora, minutos, segundos);
    }
}
```

2. Programar una clase de nombre HoraPlus, que herede de hora, y que además incluya la propiedad privada centésimas, que guarde las centésimas de segundo de la hora.

Métodos de la clase (públicos):

- Método constructor por defecto, que le de valores aleatorios a las propiedades. La hora será un valor comprendido entre 0 y 23, los minutos y segundos entre 0 y 59 y las centésimas entre 0 y 99. Invocar desde este constructor al constructor de la clase padre
- Método constructor al que se le pasen como argumentos 4 enteros y se los asigne a las propiedades. Si la hora pasada es incorrecta pondrá la hora a 0:00:00:00. Invocar al constructor de la clase padre.
- Métodos set y get para la nueva propiedad.
- visualizaHora que sobrecargue al método de la clase Hora y devuelva un String con la hora en el formato: hh:mm:ss:cc

```
import java.util.Random;

public class HoraPlus extends Hora {

    private int centesimas;
```



```
public HoraPlus() {
    super();
    Random r = new Random();
    this.centésimas = r.nextInt(100);
}

public HoraPlus(int hora, int minutos, int segundos, int centésimas) {
    super(hora, minutos, segundos);

    if (hora < 0 || hora > 23 || minutos < 0 || minutos > 59
        || segundos < 0 || segundos > 59 || centésimas < 0 ||
centésimas > 99) {
        this.hora = 0;
        this.minutos = 0;
        this.segundos = 0;
        this.centésimas = 0;
    } else {
        this.centésimas = centésimas;
    }
}

public int getCentésimas() {
    return centésimas;
}

public void setCentésimas(int centésimas) {
    this.centésimas = centésimas;
}

// Sobrecarga del método visualizaHora
@Override
public String visualizaHora() {
    return String.format("%02d:%02d:%02d:%02d", hora, minutos, segundos,
centésimas);
}
}
```

3. Hacer un programa en el que se declare un objeto de la clase Hora, creándolo utilizando el constructor por defecto, y visualice la hora que contiene el objeto. Declarar otro objeto de la clase HoraPlus, construyéndolo utilizando el constructor al que se le pasan parámetros. Visualizar la hora almacenada. Darle nuevos valores a la hora del objeto, y volver a visualizar la hora.

```
public class Main {
    public static void main(String[] args) {

        // Objeto Hora con constructor por defecto
        Hora h1 = new Hora();
        System.out.println("Hora aleatoria: " + h1.visualizaHora());

        // Objeto HoraPlus con parámetros
        HoraPlus hp1 = new HoraPlus(12, 30, 45, 50);
        System.out.println("HoraPlus inicial: " + hp1.visualizaHora());

        // Cambiar valores
        hp1.setHora(20);
        hp1.setMinutos(10);
    }
}
```



```
        hp1.setSegundos(5);  
        hp1.setCentesimas(99);  
  
        System.out.println("HoraPlus modificada: " + hp1.visualizaHora());  
    }  
}
```

4. Programar una clase de nombre Llamada que guardará la información relativa a una llamada telefónica. Tendrá las siguientes propiedades y métodos:

Propiedades: (todas ellas privadas)

String numeroTelefono;
Hora comienzoLlamada;
Hora finLlamada;

Métodos de la clase (públicos):

- Método constructor por defecto que no haga nada.
- Método constructor que reciba tres argumentos de tipo (String, Hora, Hora) y se los asigne a las propiedades de la clase.
- Métodos set y get para las propiedades de la clase.
- Método de nombre duracionLlamada que devuelva en segundos la duración de la llamada. Si la hora de finalización es inferior a la hora de comienzo de la llamada el método devolverá -1.

```
public class Llamada {  
  
    private String numeroTelefono;  
    private Hora comienzoLlamada;  
    private Hora finLlamada;  
  
    public Llamada() {  
    }  
  
    public Llamada(String numeroTelefono, Hora comienzoLlamada, Hora  
finLlamada) {  
        this.numeroTelefono = numeroTelefono;  
        this.comienzoLlamada = comienzoLlamada;  
        this.finLlamada = finLlamada;  
    }  
  
    public String getNumeroTelefono() {  
        return numeroTelefono;  
    }  
  
    public void setNumeroTelefono(String numeroTelefono) {  
        this.numeroTelefono = numeroTelefono;  
    }  
  
    public Hora getComienzoLlamada() {  
        return comienzoLlamada;  
    }  
  
    public void setComienzoLlamada(Hora comienzoLlamada) {  
        this.comienzoLlamada = comienzoLlamada;  
    }  
}
```



```

    }

    public Hora getFinLlamada() {
        return finLlamada;
    }

    public void setFinLlamada(Hora finLlamada) {
        this.finLlamada = finLlamada;
    }

    public int duracionLlamada() {
        int inicio = comienzoLlamada.getHora() * 3600
            + comienzoLlamada.getMinutos() * 60
            + comienzoLlamada.getSegundos();

        int fin = finLlamada.getHora() * 3600
            + finLlamada.getMinutos() * 60
            + finLlamada.getSegundos();

        if (fin < inicio) {
            return -1;
        }

        return fin - inicio;
    }
}

```

5. Hacer un programa en el que se declare un objeto de la clase Llamada. Darle valores a las propiedades que representen una duración válida. Escribir a continuación todos los valores del objeto, y la duración de la llamada. Cambiar los valores del objeto para que represente una llamada inválida y volver a escribir los valores del objeto.

```

public class TestLlamada {

    public static void main(String[] args) {

        Hora inicio = new Hora(10, 0, 0);
        Hora fin = new Hora(10, 5, 30);

        Llamada llamada = new Llamada("123456789", inicio, fin);

        System.out.println("Número: " + llamada.getNumeroTelefono());
        System.out.println("Inicio: " + llamada.getComienzoLlamada().visualizaHora());
        System.out.println("Fin: " + llamada.getFinLlamada().visualizaHora());
        System.out.println("Duración: " + llamada.duracionLlamada() + " segundos");

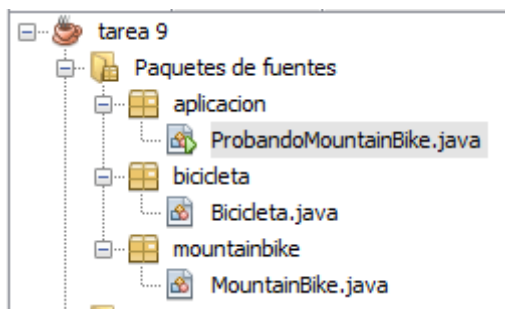
        // Llamada inválida
        llamada.setFinLlamada(new Hora(9, 0, 0));

        System.out.println("\nLlamada inválida:");
        System.out.println("Inicio: " + llamada.getComienzoLlamada().visualizaHora());
        System.out.println("Fin: " + llamada.getFinLlamada().visualizaHora());
        System.out.println("Duración: " + llamada.duracionLlamada());
    }
}

```



Cada una de las clases de los tres siguientes ejercicios tiene que estar en un paquete diferente. Darles a los paquetes, y a las clases el mismo nombre que la estructura de la imagen:



6. Programar una clase de nombre Bicicleta que nos permita describir las propiedades y el comportamiento básico de una bicicleta. Será la base de partida que nos permitirá, añadiendo otras clases, perfeccionar su comportamiento ante los avances técnicos que se produzcan. Tendrá las siguientes propiedades y métodos:

Propiedades (privadas):

- tamañoRueda: tendrá los valores 1, 2 y 3 respectivamente para los tamaños de 26, 27.5 y 29 pulgadas de diámetro. Una pulgada equivale a 2,54 cm.
- cadencia: número de pedaladas por minuto que da el ciclista.
- numPlatos: indica cuantos platos tiene la bicicleta.
- numPinones: indica cuantos piñones tiene la bicicleta.

Métodos de la clase (públicos):

- Método constructor por defecto, que no haga nada.
- Método constructor al que se le pasen como argumentos los cuatro valores que necesita una bicicleta para su definición, y se los asigne a las propiedades.
- Métodos set y get para todas las propiedades.

SOLUCIÓN:

```
package bicicleta;

public class Bicicleta {

    private int tamañoRueda;
    private double velocidad;
    private int numPlatos;
    private int numPinones;

    public Bicicleta() {
```



```
}

    public Bicicleta(int tamanoRueda, double velocidad, int numPlatos, int
numPinones) {
        this.tamanoRueda = tamanoRueda;
        this.velocidad = velocidad;
        this.numPlatos = numPlatos;
        this.numPinones = numPinones;
    }

    public int getTamanoRueda() {
        return tamanoRueda;
    }

    public void setTamanoRueda(int tamanoRueda) {
        this.tamanoRueda = tamanoRueda;
    }

    public double getVelocidad() {
        return velocidad;
    }

    public void setVelocidad(double velocidad) {
        this.velocidad = velocidad;
    }

    public int getNumPlatos() {
        return numPlatos;
    }

    public void setNumPlatos(int numPlatos) {
        this.numPlatos = numPlatos;
    }

    public int getNumPinones() {
        return numPinones;
    }

    public void setNumPinones(int numPinones) {
        this.numPinones = numPinones;
    }

    public void acelera(double v) {
        velocidad += v;
    }

    public void frena(double v) {
        velocidad -= v;
    }
}
```

7. Programar una clase de nombre MountainBike, que herede de Bicicleta, y que además añada las siguientes propiedades y métodos:

Propiedades (privadas):

- numDientes: array de 15 enteros que almacenará en sus tres primeras posiciones el número de dientes de cada uno de los tres posibles platos que puede tener una bicicleta de montaña. En las restantes posiciones se almacenarán el número de dientes de los piñones que tenga la bicicleta. En



el caso de tener menos platos o piñones solo tendrán valores las posiciones necesarias del array, el resto almacenará un 0.

- plato: indica qué plato tiene seleccionado la bicicleta, siendo un valor entre 1 y 3.
- pinon: indica qué piñón tiene seleccionado la bicicleta, siendo un valor entre 1 y 12.

Métodos de la clase (públicos):

- Método constructor por defecto, que no haga nada.
- Método constructor al que se le pasen como argumentos los 7 valores que definen una MountainBike y se los asigne a las propiedades. Invocar al constructor de la clase padre para asignar las cuatro primeras propiedades.
- Métodos set y get para las nuevas propiedades.
- Método cmAvanza, que devuelva el número de centímetros que avanza la bicicleta por cada pedalada completa que da el ciclista, en función del plato y piñón seleccionado y el tamaño de la rueda. Para calcular las vueltas que da la rueda por pedalada, hay que dividir el número de dientes del plato entre el número de dientes del piñón.

Si, por ejemplo un objeto tiene los siguientes valores:

tamanoRueda: 1

numPlatos: 2

numPinones:10

numDientes: 22-36-xx-11-13-15-17-19-21-24-28-32-36-xx-xx (xx podría tener cualquier valor, ya que en este objeto no se utilizan)

plato: 1

pinon: 10

en cada vuelta avanzará 126,78 cm

- velocidad: devuelve la velocidad a la que se mueve la bicicleta en Km/h
- visualizaMountainBike: visualizará en pantalla los datos del objeto con el formato (información del epígrafe anterior):

Diámetro de la rueda: 26 pulgadas

Número de platos: 2

Número de piñones: 10

Número de dientes de los 2 platos: 22-36 (si solo hay uno visualizar "Número de dientes del plato")

Número de dientes de los 10 piñones: 11-13-15-17-19-21-24-28-32-36 (misma consideración que para los platos)

Plato seleccionado: 1, de 22 dientes

Piñón seleccionado: 10, de 36 dientes

SOLUCIÓN:

```
package mountainbike;
```

```
public class MountainBike extends bicicleta.Bicicleta {
```

```
    private int[] numDientes = new int[15];
```




```
private int plato;
private int pinon;

public MountainBike() {
}

public MountainBike(int tamanoRueda, double velocidad, int numPlatos, int
numPinones, int numDientes[], int plato, int pinon) {
    super(tamanoRueda, velocidad, numPlatos, numPinones);
    System.arraycopy(numDientes, 0, this.numDientes, 0,
numDientes.length);
    this.plato = plato;
    this.pinon = pinon;
}

public int[] getNumDientes() {
    return numDientes;
}

public void setNumDientes(int[] numDientes) {
    System.arraycopy(numDientes, 0, this.numDientes, 0,
numDientes.length);
}

public int getPlato() {
    return plato;
}

public void setPlato(int plato) {
    this.plato = plato;
}

public int getPinon() {
    return pinon;
}

public void setPinon(int pinon) {
    this.pinon = pinon;
}

public double cmAvanza() {
    double radio;
    switch (getTamanoRueda()) {
        case 1:
            radio = 26 * 2.54 / 2;
            break;
        case 2:
            radio = 27.5 * 2.54 / 2;
            break;
        case 3:
            radio = 29 * 2.54 / 2;
            break;
        default:
            throw new AssertionError();
    }
    return ((double) numDientes[plato - 1] / numDientes[pinon + 2]) * 2 *
radio * Math.PI;
}

public void visualizaMountainBike() {
```



```

        System.out.print("Diámetro de la rueda: ");
        switch (getTamanoRueda()) {
            case 1:
                System.out.println("26 pulgadas");
                break;
            case 2:
                System.out.println("27.5 pulgadas");
                break;
            case 3:
                System.out.println("29 pulgadas");
                break;
            default:
                throw new AssertionError();
        }
        System.out.println("Número de platos: " + getNumPlatos());
        System.out.println("Número de piñones: " + getNumPinones());
        if (getNumPlatos() == 1) {
            System.out.println("Número de dientes del plato: " +
numDientes[0]);
        } else {
            System.out.print("Número de dientes de los " + getNumPlatos() + "
platos: ");
            System.out.print(numDientes[0]);
            for (int i = 1; i < getNumPlatos(); i++) {
                System.out.print("-" + numDientes[i]);
            }
            System.out.println("");
        }

        if (getNumPinones() == 1) {
            System.out.println("Número de dientes del p'ñón: " +
numDientes[3]);
        } else {
            System.out.print("Número de dientes de los " + getNumPinones() +
" piñones: ");
            System.out.print(numDientes[3]);
            for (int i = 4; i < getNumPinones() + 3; i++) {
                System.out.print("-" + numDientes[i]);
            }
            System.out.println("");
        }
        System.out.println("Plato seleccionado: " + plato + ", de " +
numDientes[plato - 1] + " dientes");
        System.out.println("Piñón seleccionado: " + pinon + ", de " +
numDientes[pinon + 2] + " dientes");
    }
}

```

8. Hacer un programa en el que se declare un objeto de la clase MountainBike, asignándole al objeto los valores:

```

tamanoRueda: 1
cadencia: 75
numPlatos: 1
numPinones:11
numDientes: 22-0-0-11-13-15-17-19-21-24-28-32-36-42-0
plato: 1

```



Visualizar los datos de esta bicicleta, y los centímetros que avanza en cada pedalada.

Modificar los siguientes valores del objeto:

tamanoRueda: 3

numPlatos: 2

numPinones:11

numDientes: 22-36-0-11-13-15-17-19-21-24-28-32-36-42-0

plato: 2

pinon: 1

Visualizar los datos de esta bicicleta con los valores iniciales y los cambios efectuados. Visualizar también los centímetros que avanza en cada pedalada y la velocidad que iría con la configuración almacenada.

Ejemplo de la ejecución del programa:

Valores de la bicicleta con los datos iniciales:

Diámetro de la rueda: 26 pulgadas

Número de platos: 1

Número de piñones: 11

Número de dientes del plato: 22

Número de dientes de los 11 piñones: 11-13-15-17-19-21-24-28-32-36-42

Plato seleccionado: 1, de 22 dientes

Piñón seleccionado: 5, de 19 dientes

En cada pedalada la bicicleta avanza 240.2293228709231 cm

La bicicleta se mueve a una velocidad de 10.81031952919154 Km/h

Valores de la bicicleta una vez modificados algunos valores:

Diámetro de la rueda: 29 pulgadas

Número de platos: 2

Número de piñones: 11

Número de dientes de los 2 platos: 22-36

Número de dientes de los 11 piñones: 11-13-15-17-19-21-24-28-32-36-42

Plato seleccionado: 2, de 36 dientes

Piñón seleccionado: 1, de 11 dientes

En cada pedalada la bicicleta avanza 757.34088500757 cm

La bicicleta se mueve a una velocidad de 34.08033982534065 Km/h

BUILD SUCCESSFUL (total time: 0 seconds)

SOLUCIÓN:

```
package aplicacion;
```

```
import mountainbike.MountainBike;
```



```
public class ProbandoMountainBike {

    public static void main(String[] args) {
        int[] numDientes = {22, 36, 0, 11, 13, 15, 17, 19, 21, 24, 28, 32,
36, 42, 0};

        //objeto utilizando el constructor con parámetros
        MountainBike orbea;
        orbea = new MountainBike(3, 18.5, 2, 11, numDientes, 2, 1);

        System.out.println("Objeto      utilizando      el      constructor
parametrizado\n");

        orbea.visualizaMountainBike();
        System.out.println("\nEn  cada  pedalada  la  bicicleta  avanza  " +
orbea.cmAvanza() + " cm");

        //objeto utilizando el constructor por defecto
        System.out.println("\nObjeto      utilizando      el      constructor      por
defecto\n");
        MountainBike mmr = new MountainBike();

        //modificamos el vector a los nuevos valores
        numDientes[0] = 22;
        numDientes[1] = 0;

        mmr.setTamanoRueda(1);
        mmr.setVelocidad(7.23);
        mmr.setNumPlatos(1);
        mmr.setNumPinones(11);
        mmr.setNumDientes(numDientes);
        mmr.setPlato(1);
        mmr.setPinon(5);

        mmr.visualizaMountainBike();
        System.out.println("\nEn  cada  pedalada  la  bicicleta  avanza  " +
mmr.cmAvanza() + " cm");
    }
}
```